BÀI TẬP CHƯƠNG 2: KIẾN TRÚC TẬP LỆNH (MIPS)

BÀI 5

5.1 & 5.2.

Chuỗi a: 1010 1110 0000 1011 0000 0000 0000 0100

Opcode : 101011 = 2b_hex --> lệnh sw --> I_type

rs: 10000 -> $s0

rt: 01011 = -> $t3

Immediate: 0000 0000 0000 0100 = 4

Hợp ngữ MIPS: sw $t3, 4($s0)

Chuỗi b: 1000 1101 0000 1000 0000 0000 0100 0000

Opcode: 100011 = 23_hex --> lệnh lw --> I_type

rs: 01000 = 8 --> $t0.

rt: 01000 = 8 --> $t0.

Immediate: 0000 0000 0100 0000 = 64

Hợp ngữ MIPS: lw $t0, 64($t0)

5.3.

Chuỗi a: 1010 1110 0000 1011 0000 0000 0000 0100 = 0xAE0B0004

Chuỗi b: 1000 1101 0000 1000 0000 0000 0100 0000 = 0x8D080040

5.4, 5.5 & 5.6.

a) Lệnh: add $t0, $t0, $zero

--> R-type

opcode = 000000

rs = $t0 = 01000

rt = $zero = 00000

rd = $t0 = 01000

shamt = 00000

funct = 100000

Mã nhị phân: 0000 0001 0000 0000 0100 0000 0010 0000

Mã HEX: 0x01004020

b) Lệnh: lw $t1, 4($s3)

--> I-type

opcode = 100011

rs = $s3 = 10011

rt = $t1 = 01001

immediate = 0000000000000100 (4_{10}) = 0x0004

Mã nhị phân: 1000 1110 0110 1001 0000 0000 0000 0100

Mã HEX: 0x8E690004

BÀI 7

Cho các giá trị thanh ghi $t0:

a: $t0 = 1010 1101 0001 0000 0000 0000 0000 0010

b: $t0 = 1111 1111 1111 1111 1111 1111 1111 1111

7.1.

Thanh ghi $t1 = 0011 1111 1111 1000 0000 0000 0000 0000

```
slt  $t2, $t0, $t1
beq  $t2, $zero, ELSE
j    DONE
ELSE: addi $t2, $zero, 2
```

DONE:

a ($t0 = 0xAD100002):

Kết quả: $t2 = 1

b ($t0 = 0xFFFFFFFF = -1)

Kết quả: $t2 = 1

7.2

a $t0 = 0xAD100002

$t0 < X luôn luôn ĐÚNG

Kết luận: Bất kỳ giá trị X 16-bit nào

b $t0 = 0xFFFFFFFF = -1

Kết luận: X số nguyên ko âm là được (16 bit)

7.3.

PC = 0x00000020 --> PC + 4 = 0x00000024

Lệnh j (Jump):

Địa chỉ đích của lệnh j được tạo bằng cách ghép 4 bit cao của PC + 4 với 26 bit địa chỉ trong lệnh (dịch trái 2 bit).

4 bit cao của PC + 4 là 0000 (dải địa chỉ nhảy tới được: 0x00000000 đến 0x0FFFFFFF).

a 0xAD100002: 4 bit cao là 1010

--> KHÔNG THỂ dùng lệnh j.

b (0xFFFFFFFF): 4 bit cao là 1111

-> KHÔNG THỂ dùng lệnh j.

Lệnh beq

Lệnh beq có immediate 16 bit [-32768, 32767]

Khoảng cách từ 0x00000024 tới 0xAD100002 hoặc 0xFFFFFFFF quá lớn, vượt xa dải biểu diễn 16-bit có dấu.

Kết luận: KHÔNG THỂ dùng lệnh beq cho cả 2 trường hợp.

BÀI 8

A) 0x02125824
B) 0x000D49C0
C) 0x22680019
D) 0x2268FFE7
E) 0x8E080018
F)0x8E08FFE8
G) 0xAC0A0030
H) 0xAC0AFFD0

BÀI 9

A)
and $t0, $t1,$s0
B)
opcode = 001000 = 8 --> addi

rs = 01001 = 9

rt = 01000 = 8

imm = 1111 1111 1111 0011 = -13

--> addi $t0,$t1, -13
C)
opcode = 101011 --> sw

rs = 01001 = 9

rt = 01000 = 8

imm = 1111 1111 1111 1100 = -4

--> sw $t0, -4($t1)

BÀI 10

A)

Từng bước thực thi:

and $t0, $s0,$s1:

0x12345678 & 0x00000007 = 0x00000000

$t0 = 0x00000000

or $t1, $s0,$s1:

0x12345678 | 0x00000007 = 0x1234567F

$t1 = 0x1234567F

nor $t0, $t0,$t1:

$t0 = 0xEDCBA980

sll $t0,$t0, 3:

Dịch trái 3 bit: 0xEDCBA980 = 0x6E5D4C00

Kết quả: $s0 = 0x12345678

$s1 = 0x00000007

$t0 = 0x6E5D4C00

$t1 = 0x1234567F

B)

Cho `$s0 = 0x0000000F

andi $t0, $s0, 12:

$t0 = 15 & 12 = 0x0000000C

nor $t0, $t0, $zero:

$t0 = ~(0x0000000C | 0) = ~(0x0000000C) = 0xFFFFFFF3

ori $t0, $t0, 3:

$t0 = 0xFFFFFFF3 | 0x00000003 = 0xFFFFFFF3

srl $t0, $t0, 2:

Dịch phải logic 2 bit: 0xFFFFFFF3 = 0x3FFFFFFC

Kết quả sau khi chạy:

$s0 = 0x0000000F

$t0 = 0x3FFFFFFC

C)Từng bước thực thi:

slt $t2, $t0, $t1:

So sánh có dấu: $143 < 159 là ĐÚNG. -->$t2 = 1.

beq $t2, $zero, ELSE:

Vì `$t2 = 1 , rẽ nhánh không xảy ra.

add $t2,$t2, $t0L:

$t2 = 1 + $t0 = 1 + 0x0000008F = 0x00000090

j DONE:

Nhảy tới DONE.

Kết quả sau khi chạy:

$t2 = 0x00000090

D)

Ban đầu: $s0 = 2, $t1 = 6.

Vòng lặp loop thực hiện 6 lần

Mỗi lần lặp thực hiện sll $s0, $s0, 1, tương đương nhân giá trị $s0 với 2

Giá trị $s0 qua từng vòng lặp:

Lần 1: $s0 = 2 * 2 = 4

Lần 2: $s0 = 4 * 2 = 8

Lần 3: $s0 = 8 * 2 = 16

Lần 4: $s0 = 16 * 2 = 32

Lần 5: $s0 = 32 * 2 = 64

Lần 6: $s0 = 64 * 2 = 128

Sau khi $t1 = 0, thoát khỏi vòng lặp đến nhãn end

Lệnh addi $s1,$s0, 2 gán $s1 = 128 + 2 = 130

Kết quả sau khi chạy:

$s0 = 128 = 0x00000080

11)

A)

```
slt  $t0, $s0, $s1          # $t0 = 1 nếu i < j, ngược lại $t0 = 0
beq  $t0, $zero, ELSE_A     # Nếu i >= j ($t0 == 0), nhảy sang nhánh ELSE
sll  $t1, $s0, 2            # $t1 = i * 4 (tính offset byte của A[i])
add  $t2, $s3, $t1          # $t2 = địa chỉ của A[i]
lw   $t3, 0($t2)            # $t3 = A[i]
addi $t3, $t3, 1            # $t3 = A[i] + 1
sw   $t3, 0($t2)            # A[i] = A[i] + 1

addi $t4, $zero, 5          # $t4 = 5
sw   $t4, 4($t2)            # A[i+1] = 5 (địa chỉ A[i+1] = $t2 + 4)
```

```
    j    EXIT_IF_A            # Hoàn thành IF, nhảy qua ELSE

ELSE_A:
    sll  $t1, $s0, 2          # $t1 = i * 4
    add  $t2, $s3, $t1        # $t2 = địa chỉ của A[i]
    lw   $t3, 0($t2)          # $t3 = A[i]
    addi $t3, $t3, -1         # $t3 = A[i] - 1
    sw   $t3, 0($t2)          # A[i] = A[i] - 1

    addi $t4, $zero, 10       # $t4 = 10
    sw   $t4, 4($t2)          # A[i+1] = 10 (địa chỉ A[i+1] = $t2 + 4)

EXIT_IF_A:
    addi $s0, $s0, 1          # i++

B)      slt  $t0, $s1, $s0        # $t0 = 1 nếu j < i
    bne  $t0, $zero, ELSE_B   # Nếu j < i (điều kiện i <= j sai) -> nhảy tới ELSE
    slt  $t0, $zero, $s1      # $t0 = 1 nếu 0 < j
    beq  $t0, $zero, ELSE_B   # Nếu j <= 0 ($t0 == 0) -> nhảy tới ELSE

    sll  $t1, $s0, 2          # $t1 = i * 4
```

```
        add  $t2, $s3, $t1          # $t2 = địa chỉ của A[i]
        lw   $t3, 0($t2)            # $t3 = A[i]
        lw   $t4, 4($t2)            # $t4 = A[i+1]

        add  $t5, $t3, $t4          # $t5 = A[i] + A[i+1]

        sll  $t6, $s1, 2            # $t6 = j * 4
        add  $t7, $s3, $t6          # $t7 = địa chỉ của A[j]
        sw   $t5, 0($t7)            # A[j] = A[i] + A[i+1]
        j    EXIT_IF_B

ELSE_B:
        sll  $t1, $s0, 2            # $t1 = i * 4
        add  $t2, $s3, $t1          # $t2 = địa chỉ của A[i]
        lw   $t3, 0($t2)            # $t3 = A[i]
        lw   $t4, 4($t2)            # $t4 = A[i+1]

        sub  $t5, $t3, $t4          # $t5 = A[i] - A[i+1]

        sll  $t6, $s1, 2            # $t6 = j * 4
        add  $t7, $s3, $t6          # $t7 = địa chỉ của A[j]
        sw   $t5, 0($t7)            # A[j] = A[i] - A[i+1]
```

```
EXIT_IF_B:
        addi $s0, $s0, 1            # i++


C)
LOOP_C:
        slt  $t0, $zero, $s0        # $t0 = 1 nếu 0 < i
        beq  $t0, $zero, EXIT_WHILE_C # Nếu i <= 0 ($t0 == 0), thoát khỏi vòng lặp

        # Thân vòng lặp
        sll  $t1, $s0, 2            # $t1 = i * 4
        add  $t2, $s3, $t1          # $t2 = địa chỉ của A[i]
        lw   $t3, 0($t2)            # $t3 = A[i]

        sll  $t4, $t3, 3            # $t4 = A[i] * 8 (dịch trái 3 bit tương đương nhân với 8)
        sw   $t4, 4($t2)            # A[i+1] = A[i] * 8 (địa chỉ A[i+1] = $t2 + 4)

        addi $s0, $s0, -1           # i--
        j    LOOP_C                 # Quay lại kiểm tra vòng lặp

EXIT_WHILE_C:
        addi $t5, $zero, 5          # $t5 = 5
        sw   $t5, 0($s3)            # A[0] = 5 (địa chỉ A[0] chính là địa chỉ nền $s3)
```

D)

```
add  $s1, $s5, $zero        # j = value
        addi $s0, $zero, 1          # i = 1 (khởi tạo biến đếm vòng lặp for)

LOOP_FOR_D:
        # Kiểm tra điều kiện i < j
        slt  $t0, $s0, $s1          # $t0 = 1 nếu i < j
        beq  $t0, $zero, EXIT_FOR_D # Nếu i >= j ($t0 == 0), kết thúc
vòng lặp

        # Thân vòng lặp: A[i] = B[i]
        sll  $t1, $s0, 2            # $t1 = i * 4 (offset byte)
        add  $t2, $s4, $t1          # $t2 = địa chỉ của B[i]
        lw   $t3, 0($t2)            # $t3 = B[i]

        add  $t4, $s3, $t1          # $t4 = địa chỉ của A[i]
        sw   $t3, 0($t4)            # A[i] = B[i]

        addi $s0, $s0, 1            # i++
        j    LOOP_FOR_D             # Tiếp tục vòng lặp

EXIT_FOR_D:
        add  $s1, $zero, $zero      # j = 0
```

E)

```
add  $s1, $s5, $zero        # j = value

        add  $s4, $zero, $zero      # max = 0

        add  $s0, $zero, $zero      # i = 0 (khởi tạo biến đếm vòng lặp)


LOOP_FOR_E:

        # Kiểm tra điều kiện i < j

        slt  $t0, $s0, $s1          # $t0 = 1 nếu i < j

        beq  $t0, $zero, EXIT_FOR_E # Nếu i >= j ($t0 == 0), kết thúc vòng
lặp


        # Đọc giá trị A[i]

        sll  $t1, $s0, 2            # $t1 = i * 4

        add  $t2, $s3, $t1          # $t2 = địa chỉ của A[i]

        lw   $t3, 0($t2)            # $t3 = A[i]


        # Kiểm tra điều kiện if (A[i] > max) tương đương (max < A[i])

        slt  $t4, $s4, $t3          # $t4 = 1 nếu max < A[i]

        beq  $t4, $zero, SKIP_MAX_E # Nếu max >= A[i] ($t4 == 0), bỏ
qua việc cập nhật


        # Cập nhật max = A[i]

        add  $s4, $t3, $zero        # max = A[i]
```

```
SKIP_MAX_E:
        addi $s0, $s0, 1            # i++
        j    LOOP_FOR_E             # Tiếp tục vòng lặp

EXIT_FOR_E:
        add  $s1, $zero, $zero      # j = 0
```